

# EE 475 Introduction to Image Processing

## Homework 06

Boran Aybak Kilic  
Student ID: 2022401207

### 1 Interference on K-Space

The corrupted image contains strong periodic noise manifested as diagonal sinusoidal stripes. In image processing, the frequency domain (K-space) representation allows us to isolate such periodic patterns because they concentrate their energy at specific frequencies.

When we compute the Discrete Fourier Transform (DFT) of the image, this sinusoidal interference appears as a pair of distinct, high-energy "spikes" (conjugate impulses) located symmetrically around the center of the spectrum. This is because a pure sine wave in the spatial domain transforms into impulsive functions in the frequency domain.

As shown in Figure 1, the K-space contains the general spectral information of the astronaut, but it also contains two anomalous bright spots. Figure 2 highlights these spots. By identifying the coordinates of these high-energy bursts, we locate the exact frequency and orientation of the noise.

Additionally, a bright vertical line runs through the center of the spectrum (visible in Figure 1b). This is not part of the noise but an artifact caused by the rectangular boundary of the image. Since the DFT assumes the image is periodic (repeating infinitely), the intensity mismatch between the top and bottom edges of the image creates a sharp discontinuity (a step edge). This discontinuity generates a splash of energy along the vertical frequency axis. We ignore this line and focus on the diagonal noise spikes.

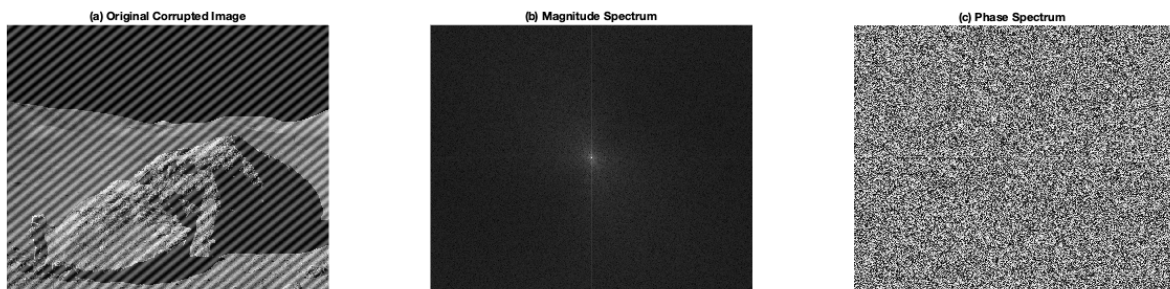


Figure 1: (a) Original image with diagonal interference. (b) Magnitude of k-space raised to the power of 0.2 for better visualisation c) Phase of the k-space

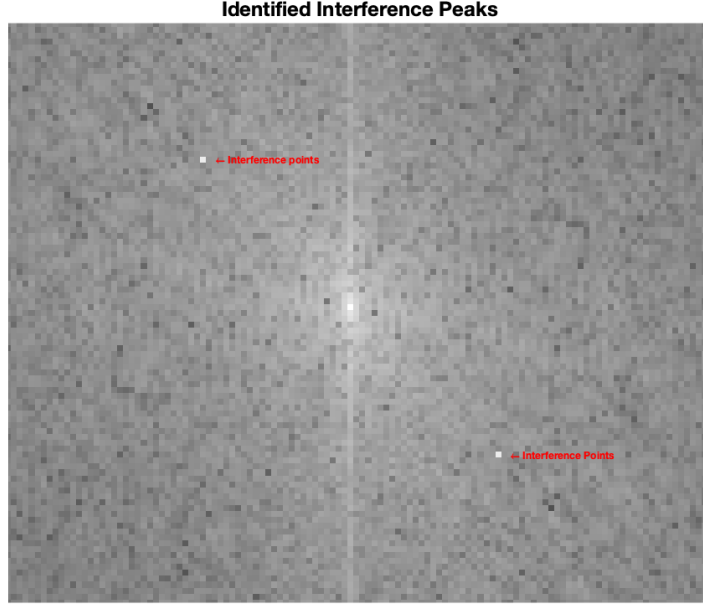


Figure 2: K-space with arrows pointing to the conjugate noise impulses.

## 2 Notch Filtering

To restore the image, we use Notch Reject Filters to suppress the specific frequencies associated with the noise while passing the rest of the image information.

### 2.1 Ideal Notch Filter

The Ideal Notch Filter uses a binary mask. We define a radius  $D_0$  around the noise coordinates  $(u_k, v_k)$ . If a frequency component falls inside this circle, it is multiplied by 0 (removed completely); otherwise, it is multiplied by 1 (preserved).

The transfer function is:

$$H(u, v) = \begin{cases} 0 & \text{if } D_k(u, v) \leq D_0 \text{ or } D_{-k}(u, v) \leq D_0 \\ 1 & \text{otherwise} \end{cases} \quad (1)$$

Figure 3 shows the result. While the periodic noise is removed, the restored image suffers from "ringing" artifacts (faint ripples near edges). This happens because the Ideal filter makes an abrupt, discontinuous cut in the frequency domain (from 1 to 0 instantly). In signal processing, a sharp box-like cutoff in the frequency domain corresponds to a sinc function ripple in the spatial domain (the Gibbs phenomenon).

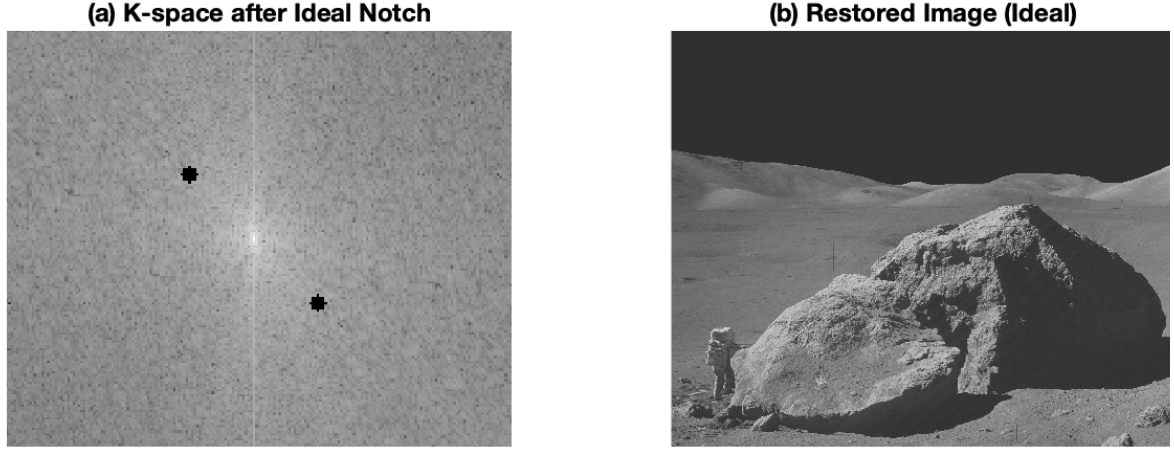


Figure 3: (a) K-space with frequencies removed by the Ideal filter. Notch radius was chosen to be 3. (b) Restored image showing slight ringing artifacts.

## 2.2 Butterworth Notch Filter

To fix the ringing issue, we use a Butterworth Notch Filter. Instead of a hard binary cutoff, this filter transitions smoothly from the passband (1) to the stopband (0).

The transfer function for a filter of order  $n$  is:

$$H(u, v) = \frac{1}{1 + \left[ \frac{D_0}{D_k(u, v)} \right]^{2n}} \cdot \frac{1}{1 + \left[ \frac{D_0}{D_{-k}(u, v)} \right]^{2n}} \quad (2)$$

As shown in Figure 4, the Butterworth filter suppresses the noise "spikes" without an abrupt edge in the K-space. This smooth transition significantly reduces the ripples in the spatial domain impulse response. Consequently, the restored image is clearer and more natural, retaining the original details with effects of artificial ringing introduced by the ideal filter reduced.

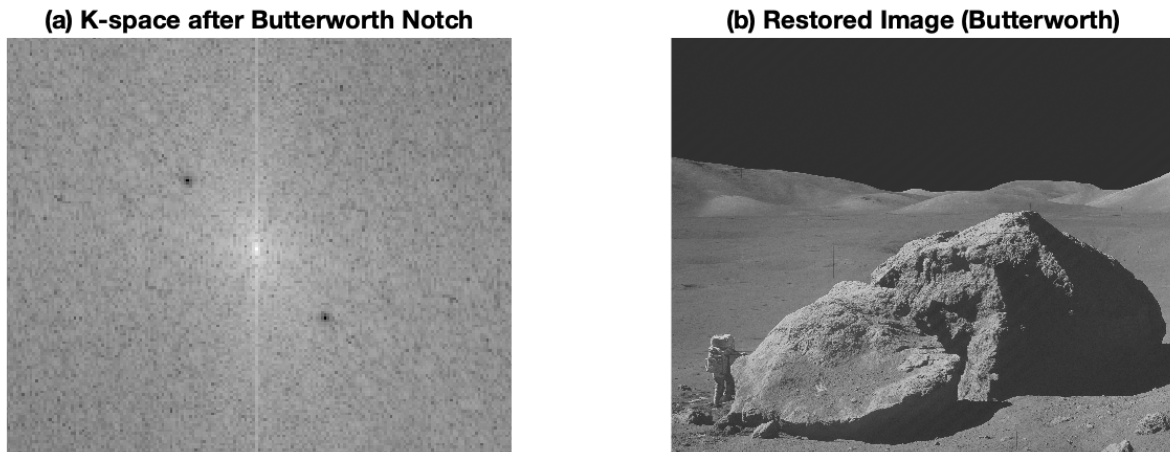


Figure 4: (a) K-space with Butterworth filter applied (note the smooth dark regions). Degree of Butterworth filter was chosen to be 2. (b) Restored image with higher quality and no visible ringing.

### 2.3 Gaussian Notch Filter

I've also implemented a Gaussian Notch Filter. Like the Butterworth filter, the Gaussian filter provides a smooth transition, but its profile is based on the exponential Gaussian function, which ensures zero ringing in the spatial domain (since the Fourier transform of a Gaussian is also a Gaussian).

The transfer function is:

$$H(u, v) = \left[ 1 - e^{-\frac{D_K(u, v)^2}{2D_0^2}} \right] \cdot \left[ 1 - e^{-\frac{D_{-K}(u, v)^2}{2D_0^2}} \right] \quad (3)$$

Figure 5 shows the result of the Gaussian filter. It provides the smoothest possible transition, effectively eliminating ringing artifacts. However, because the "notch" is not as steep as higher-order Butterworth filters, slightly more background image information might be attenuated near the noise frequencies, though in this case, the visual quality remains very high with much better attenuation of interference pattern compared to previous two cases.

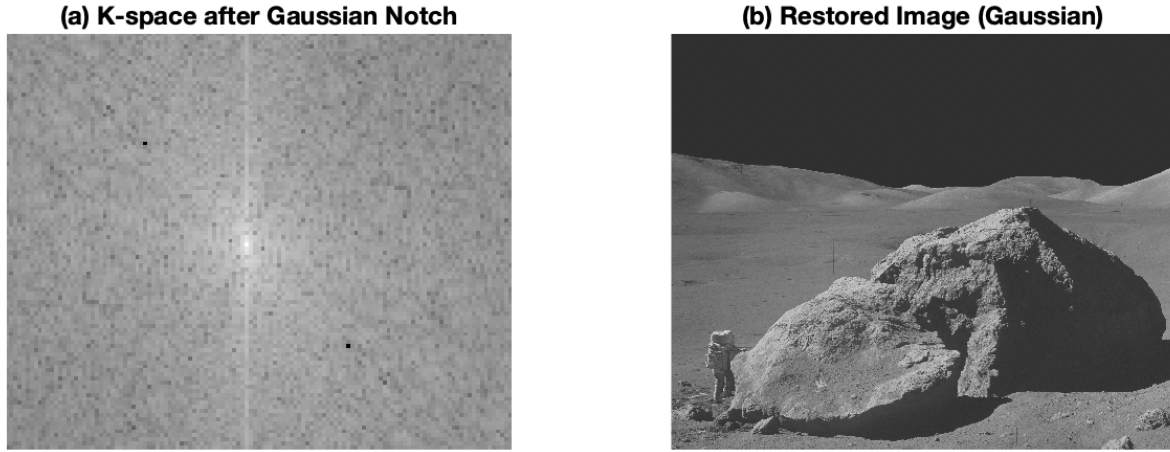


Figure 5: (a) K-space with Gaussian filter applied. Sigma of Gaussian filter was chosen to be 4. (b) Restored image showing smooth transitions and effective noise removal.

## A Code Listings

Below is the code for taking 2D-DFT of the image, design and implementation of the Notch reject filters, and plotting the results.

```
1
2 % 1. Read and Prepare Image
3 im = imread('astronaut-interference.tif');
4 im_double = double(im);
5
6 % 2. Frequency Domain Transformation by taking 2D-DFT
7 F = fft2(im_double);
8 Fsh = fftshift(F);
9 S = log(1+abs(Fsh)); % Log Magnitude for visualization ,
10 % NOTE: Plus 1 is added to log magnitude to ensure S is strictly positive
11 % for display purposes.
12
13
14 % Mask out DC component to find interference peaks
15 temp_S = abs(Fsh);
```

```

16 radius_dc = 5;
17 [M, N] = size(Fsh);
18 center_x = floor(N / 2) + 1;
19 center_y = floor(M / 2) + 1;
20 [X, Y] = meshgrid(1:N, 1:M);
21 dist_from_center = sqrt((X - center_x).^2 + (Y - center_y).^2);
22 temp_S(dist_from_center < radius_dc) = 0;
23
24 % --- Find the Top 2 Peaks corresponding to sinusoidal interference pattern ---
25 [max_vals, linear_indices] = maxk(temp_S(:), 2);
26
27 % Convert the linear indices back to row/column subscripts
28 [rows, cols] = ind2sub(size(temp_S), linear_indices);
29
30 % Define pt1 and pt2 using the results
31 % maxk returns results in descending order, so the first index is the largest
   peak
32 pt1 = [rows(1), cols(1)];
33 % The second index is the second conjugate peak
34 pt2 = [rows(2), cols(2)];
35
36 % Calculate distances to the interference points
37 dist1 = sqrt((X - pt1(2)).^2 + (Y - pt1(1)).^2);
38 dist2 = sqrt((X - pt2(2)).^2 + (Y - pt2(1)).^2);
39
40 % Common Parameters
41 D0 = 3; % Notch radius (Cutoff frequency)
42 n = 2; % Butterworth order
43 sigma = 4; % Standart deviation of Gaussian Filter
44
45 % -----
46 % 4. Filter Design
47 % -----
48
49 % A. Ideal Notch Filter
50 % H = 0 if distance <= D0, else 1
51 H_ideal = ones(M, N);
52 H_ideal(dist1 <= D0) = 0;
53 H_ideal(dist2 <= D0) = 0;
54
55 % B. Butterworth Notch Reject Filter
56 % Formula:  $H = 1 / (1 + (D0 / Dk)^{2n})$ 
57 % We multiply two highpass filters shifted to the noise coordinates.
58 % Note: Add eps to distance to avoid division by zero
59 H_but1 = 1 ./ (1 + (D0 ./ (dist1 + eps)).^(2*n));
60 H_but2 = 1 ./ (1 + (D0 ./ (dist2 + eps)).^(2*n));
61 H_butter = H_but1 .* H_but2;
62
63 % C. Gaussian Notch Reject Filter
64 H_gauss = (1 - exp(-(dist1.^2) ./ 2)) .* (1 - exp(-(dist2.^2) ./ 2));
65
66 % -----
67 % 5. Apply Filters and Reconstruct
68 % -----
69 % Apply Ideal
70 G_ideal_sh = Fsh .* H_ideal;
71 img_ideal = real(ifft2(ifftshift(G_ideal_sh)));
72 % Apply Butterworth
73 G_butter_sh = Fsh .* H_butter;
74 img_butter = real(ifft2(ifftshift(G_butter_sh)));
75 % Apply Gaussian
76 G_gauss_sh = Fsh .* H_gauss;
77 img_gauss = real(ifft2(ifftshift(G_gauss_sh)));

```

```

78
79 % -----
80 % 6. Generate Figures for Report
81 % -----
82
83 % Figure 1: Original Image and its K-space (2D-DFT)
84 f1 = figure('Name', 'Figure 1', 'Color', 'w');
85 subplot(1, 3, 1); imshow(im); title('(a) Original Corrupted Image');
86 subplot(1, 3, 2); imshow(abs(Fsh).^0.2, []); title('(b) Magnitude Spectrum');
87 subplot(1, 3, 3); imshow(angle(Fsh), []); title('(c) Phase Spectrum ');
88
89 % Figure 2: Spectrum with Arrows Pointing to Interference
90 f2 = figure('Name', 'Figure 2', 'Color', 'w');
91 imshow(S, [], 'InitialMagnification', 'fit'); title('Identified Interference
    Peaks','FontSize',24);
92 hold on;
93 text(pt1(2)+2, pt1(1), '\leftarrow Interference Points', 'Color', 'r', 'FontSize
    ', 12, 'FontWeight', 'bold');
94 text(pt2(2)+2, pt2(1), '\leftarrow Interference Points', 'Color', 'r', 'FontSize
    ', 12, 'FontWeight', 'bold');
95 hold off;
96
97
98 % Figure 3: Ideal Notch Filter Results
99 f3 = figure('Name', 'Figure 3', 'Color', 'w');
100 subplot(1, 2, 1); imshow(log(1 + abs(G_ideal_sh)), []); title('(a) K-space after
    Ideal Notch','FontSize',24);
101 subplot(1, 2, 2); imshow(uint8(img_ideal)); title('(b) Restored Image (Ideal)',
    'FontSize',24);
102
103
104 % Figure 4: Butterworth Notch Filter Results
105 f4 = figure('Name', 'Figure 4', 'Color', 'w');
106 subplot(1, 2, 1);
107 imshow(log(1 + abs(G_butter_sh)), []);
108 title('(a) K-space after Butterworth Notch','FontSize',24);
109 subplot(1, 2, 2); imshow(uint8(img_butter)); title('(b) Restored Image (
    Butterworth)','FontSize',24);
110
111 % Figure 5: Gaussian Notch Filter Results
112 f5 = figure('Name', 'Figure 5', 'Color', 'w');
113 subplot(1, 2, 1); imshow(log(1 + abs(G_gauss_sh)), []); title('(a) K-space after
    Gaussian Notch','FontSize',24);
114 subplot(1, 2, 2); imshow(uint8(img_gauss)); title('(b) Restored Image (Gaussian)
    ', 'FontSize',24);

```

Listing 1: MATLAB code to clear interference pattern on the image using Notch Reject Filters